

Learning goals: Reading and writing data in csv format, defining and manipulating data frames, introduction to 'factor' class and its usage for mixed data types, converting data frames into matrices, calculations on rows or columns of a matrix, using 'sweep()' function, applying functions with 'apply', 'lapply' and 'tapply', using the max.col function

EXERCISES

This lab will use the data used in the study described in "Survival of patients with primary brain tumors: Comparison of two statistical approaches", PLoS One, 11, e0148733 (2016) and also data from the text file "pgfull.txt".

- (1) Read in the file "BrainCancer.csv" and store the data as data frame
- (2) Print the basic features of the data including the
 - (i) dimensions of the data,
 - (ii) column names of the data,
 - (iii) row names,
 - (iv) the appearance of the first 30 lines of data, and
 - (v) the data representation as a frequency table
 - (vi) how many categorical variables are present
 - (vii) What are the categorical variables present
 - (viii) how many levels does each categorical variable have and what are the levels
- (3) Print the following statistical parameters of the data of
 - (i) mean of column describing the gross tumor volume (GTV)
 - (ii) mean of column describing the time
 - (iii) median of GTV column (how does it compare with the mean?)
 - (iv) mode of GTV column (how does it compare with median and mean? Is the distribution symmetric?)
 - (v) standard deviation of GTV column
 - (vi) statistical summary of the GTV column data
 - (vii) histogram plot of GTV data (does the appearance agree with the mean, median, mode order?)
 - (viii) skewness value of GTV data (You need to install and load the "moments" library first)
 - (ix) kurtosis value of the GTV data (do the skewness and kurtosis match with your expectations as learnt in the lecture?)
 - (x) Make a boxplot of the GTV data with default options and compare this with another boxplot of the same data with the following options: range=0.1, horizontal=FALSE, border=c("blue"), col=c("red"). Repeat the plot with range=0.2 and range=0.05. Comment on the differences.
 - (xi) Make a boxplot of 3 data sets – GTV, KI, and time. Which of the data has the broadest distribution?
- (4) Build subsets of the data below.
 - (i) GTV data with values > 20. Also print the dimensions of the subset.
 - (ii) Row numbers 1,3,8,9,13,14, 18 and 21
 - (iii) Obtain the indices of the rows that have only Female sex entries, and using this list of indices build a subset of the data
 - (iv) Extract the columns GTV and KI and build a new column that uses this formula: `data$gtv*data$ki/234`. Build a new dataframe containing the three columns GTV, KI and the newly added column.
 - (v) Write the subset data corresponding to "Female" entries (above) as a dataframe, and write the dataframe to a csv file – call this file as lab4_female_BrainCancer.csv

- (5) Manipulating the 'factors' in existing data
 - (i) Change the current class of the headers in the Brain Cancer data to the class 'factor' and check that it is indeed the factor class.
 - (ii) Print the number of levels in the category 'Sex' using `nlevels()` function
 - (iii) Print the levels in the category 'Diagnosis' using the `levels()` function
- (6) Generating factor levels using `gl()` function
 - (i) Generate a new category called 'Temperature' containing the same number of elements as number of rows in the Brain Cancer data with labels 'Hot', 'Cold' and 'lukewarm'.
 - (ii) Add this category of data to the above Brain Cancer dataframe and give it a new dataframe name. Print this new dataframe.
- (7) Using the `tapply()` function. Try the following and see what you get
 - (i) `tapply(data$gtv, data$ki, mean)`
 - (ii) `tapply(data$gtv, data$ki, mean, trim=0.1)` What does `trim` option do here?
- (8) Finding parallel-max and min. Use the `pmin()` and `pmax()` functions to find the minimum for each triplet set of 3 vectors `data$gtv`, `data$ki` and `data$time`
- (9) Difference between `rank()`, `sort()` and `order()`.
 - (i) Store the output of the 3 functions with input data `data$gtv` as vectors `ranks`, `sorted` and `ordered`. Create a new dataframe with the following columns: `data$gtv`, `ranks`, `sorted` and `ordered` and print this dataframe.
 - (ii) Print the diagnosis column using the indices from the `ordered` vector and create a new dataframe with the `data$gtv` and `data$diagnosis` columns. Write this dataframe to a new csv file called 'lab4_ordered_data.csv'.
- (10) Converting data frames to matrices. This is important to do efficient numerical calculations with the data.
 - (i) Extract the following rows and columns from the Brain cancer data: rows 1-6, columns 3-8.
 - (ii) Convert the class of this object into a matrix using `as.matrix()` function and check the `attributes` to make sure it is a matrix
 - (iii) Create a `newcol` vector/column by adding 3 vectors - `data$ki`, `data$gtv`, `data$time`.
 - (iv) Create a new dataframe by adding this column to the last, call this new dataframe as `newcoladded`. Print the column names and make sure that `newcol` has been added.
 - (v) One can add columns in another way, using `cbind()`. Use `cbind()` to add `newcol` to the original `data` dataframe. Call this new dataframe `newcoladded2`. Print the column names to make sure the new column has been added.
 - (vi) Pick rows 26 and 35 from the original data and add these as new rows to the original data using the `data.frame()` function. Print the new dataframe and make sure that the rows have been added. Also print the dimensions of the newly created dataframe.
- (11) Adding row and column names. Create a 4x5 matrix with the following data. (1,0,2,5,3,1,1,3,1,3,3,1,0,2,2,1,0,2,1,0). Then add rownames to this matrix using the `rownames` command where the row numbers are prefixed by 'Trial-'. Similarly, add column names to this matrix by (a) supplying 5 different names as "aspirin", "paracetamol", "nurofen", "hedex", "placebo", and also by (b) using the `dimnames()` function where each column number is prefixed by "drug."
- (12) Calculations on rows or columns of the matrix. Use the matrix defined in the previous problem to do the following. Predict your answer first before you print the answer.
 - (i) `mean(X[,5])`
 - (ii) `var(X[4,])`

- (iii) `rowSums(X)` calculates the sum of elements along each row. Faster method: `apply(X,1,sum)`
- (iv) `colSums(X)` Faster method: `apply(X,2,sum)`
- (v) `rowMeans(X)` alternative: `apply(X,1,mean)`
- (vi) `colMeans(X)` alternative: `apply(X,2,mean)`
- (vii) Sum groups of rows within a column. `rowsum(X,c("A","B","B","A"))`. Another method is to use the `tapply()` or `aggregate()` function:
 - (a) `group=c("A","B","B","A")`
 - (b) what do `row(X)` and `col(X)` do? As an alternative to `rowsum` do the following.
 - (c) `tapply(X,list(group[row(X)], col(X))),sum)`
 - (d) `aggregate(X,list(group),sum)`
- (viii) Shuffling the elements of a column: `apply(X,2,sample)`. How do we shuffle elements of a row?
- (ix) Add a row at the bottom of matrix `X` showing the column means, and a column at the right showing the row variances. `X <- rbind(X, apply(X,2,mean))` will add the row at the bottom, and `X <- cbind(X,apply(X,1,var))` will add a column at the right showing the row variances. Also add headings to the last column and row as follows: `headings <- c(paste("drug.",1:5,sep=""),"var")` followed by `dimnames(X)<- list(NULL,headings)` and `headings <- c(paste("Trial-",1:4,sep=""),"Mean")` followed by `rownames(X) <- headings`. Print the `X` matrix now to make sure all the above changes have been made, and it looks as thus:

	drug.1	drug.2	drug.3	drug.4	drug.5	var
Trial-1	1	3	1	0.00	0.00	1.50000
Trial-2	0	1	3	2.00	2.00	1.30000
Trial-3	2	1	3	2.00	1.00	0.70000
Trial-4	5	3	1	1.00	0.00	4.00000
Mean	2	2	2	1.25	0.75	0.33125

- (13) The `sweep()` function is used to 'sweep out' summaries from vectors, matrices, arrays or dataframes. To understand how this function works, carry out the following steps.
 - (i) Create a dataframe containing the columns `data$ki`, `data$gtv` and `data$time`. Call this dataframe as `eg_sweep`.
 - (ii) Next we need to create a vector containing the parameters that you intend to sweep out. For example let's say we want column means. Use the `apply()` function to obtain a vector called `cols` that contains the mean of each column.
 - (iii) Let's first do the sweeping operation without using `sweep()` function, and then use `sweep()` to perform the same thing. Execute the following two commands:


```
cols.means <- matrix(rep(cols,rep(dim(eg_sweep)[1],dim(eg_sweep)[2])),
nrow=dim(eg_sweep)[1])
print(cols.means)
eg_sweep_alt <- eg_sweep - cols.means
print("Method 1")
print(eg_sweep_alt)
```
 - (iv) Now apply the `sweep()` function:


```
eg_sweep1 <- sweep(eg_sweep,2,cols)
#columns will be departures from relevant col means
print("Method 2")
print(eg_sweep1)
```

- (14) Using the `max.col` function. Read the data present in `pgfull.txt`. Extract columns 1:54 from the read data and call the subset data as `species`. `max.col(species)` returns the column index of the max value in each row; print this. We will use these indices to collect the names of the species as `names(species)[max.col(species)]`. We then use the `table` function to build a frequency table of each of the species as `table(names(species)[max.col(species)])`. It turns out that there is no `min.col()` function in R, and one needs an alternative. How can `max.col` be used to extract the column indices that give the minimum value along each row?
- (15) Introduction to Lists. Execute the following to learn the basics of lists.
- (i) Lists can handle different lengths of arrays and also mixed data types.

```
apples <- c(4,4.5,4.2, 5.1,3.9)
oranges <- c(TRUE,TRUE,FALSE)
chalk <- c("limestone","marl","oolite","CaCO3")
cheese <- c(3.2-4.5i,12.8+2.2i)
items <- list(apples, oranges, chalk, cheese)
print(items)
#subscripts on lists have double square brackets
print(items[[3]])
print(items[[3]][3])
```

What happens if you try to create a dataframe of the 4 objects given above? Also does `items[3]` work the same as `items[[3]]` ? What's the difference?
 - (ii) Lists can be indexed using the '\$' if the subsets have names associated with them.

```
print(names(items))
items <- list(first=apples,second=oranges,third=chalk,fourth=cheese)
print(items$fourth)
print(class(items))
```
 - (iii) The function `lapply()` is used to apply functions to lists:

```
print(lapply(items,length))
print(lapply(items,class))
print(lapply(items,mean)) #Predict the output first
```
 - (iv) The `summary()` function allows us to output the name of the sublists, length and mode of the data present. On the other hand the `str()` (structure) function is the most useful for lists, and outputs the names, mode, data entries as well as the lengths of each sublist.